

SIG-RECETTE

Structure, Procédure & Flux des Paiements

Laravel 10 · Livewire 3 · Symfony Workflow
Espèces · Chèque · Mobile Money (QOSIC / FedaPay / PayGate)

SIG-RECETTE — Structure, Procédure & Flux des Paiements

Public : Développeurs, intégrateurs, agents comptables et administrateurs de projet de SIG-RECETTE. **Périmètre :** Description complète du domaine paiement — modèle de données, machine d'état, services, intégration mobile-money, synchronisation, notifications et procédures de bout en bout. **Stack :** Laravel 10 / PHP 8.2 · Livewire 3 · Symfony Workflow (zerodahero/laravel-workflow) · Spatie Permissions (chaînes françaises) · Barryvdh DomPDF.

Table des matières

1. Vue d'ensemble
2. Glossaire métier
3. Architecture générale
4. Modèle de données
5. Énumérations
6. Machine d'état des avis (workflow)
7. Cycle de vie d'un paiement
8. Procédure de paiement en espèces ou par chèque
9. Procédure de paiement mobile / numérique
10. Suivi des soldes par code taxe
11. Génération de reçus et de PDF
12. Synchronisation avec l'application de collecte distante
13. Événements, écouteurs, jobs et notifications
14. Autorisations et permissions
15. Référence des routes
16. Référence de configuration
17. Référence des migrations de base de données
18. Procédures étape par étape
19. Résolution des problèmes

1. Vue d'ensemble

SIG-RECETTE gère l'**émission des avis de mise en recouvrement (Invoice)** et la **collecte des paiements (Payment)** pour une régie de recettes publiques. Trois canaux de paiement sont pris en charge :

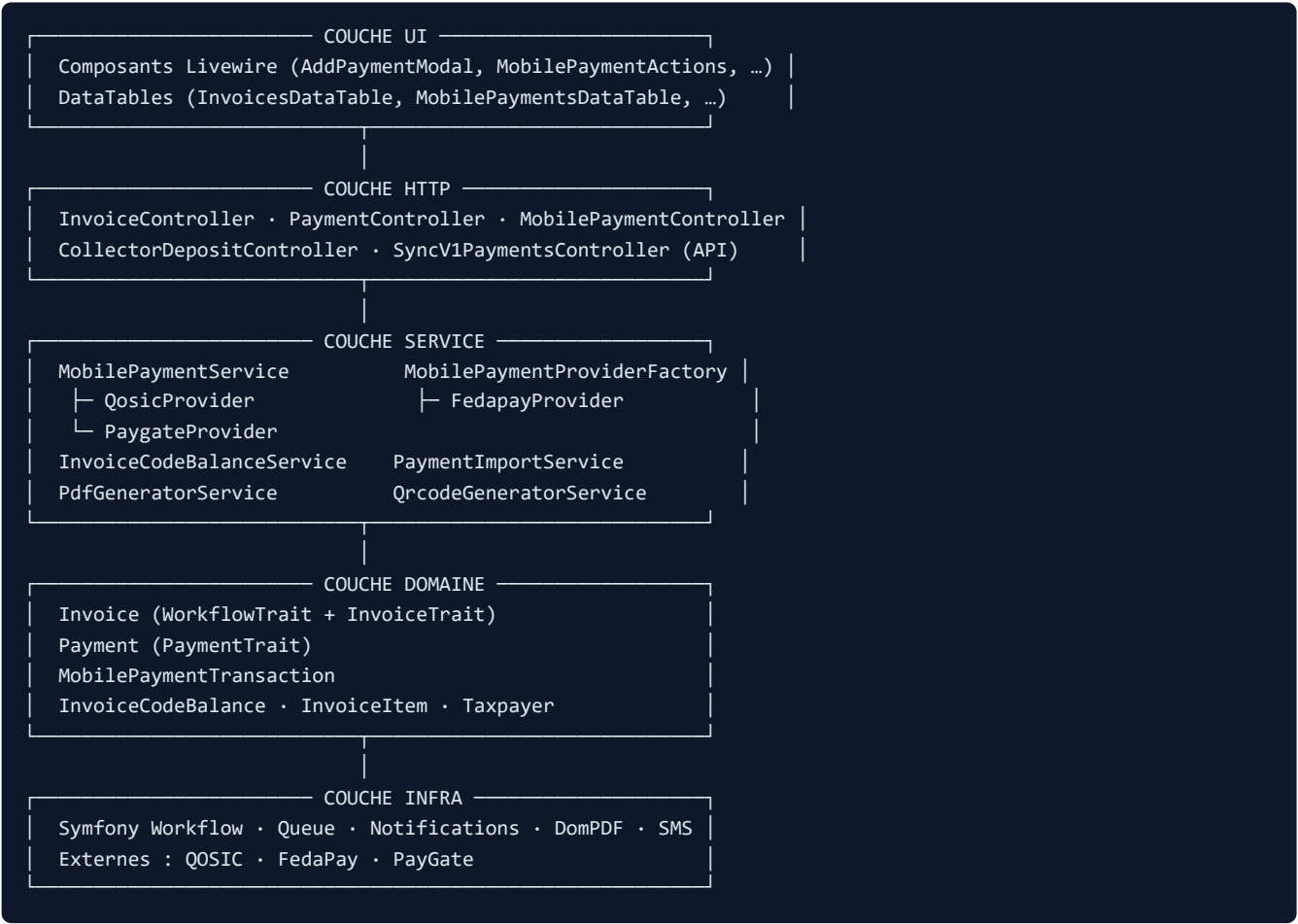
Canal	Code	Description
Espèces	CASH	Paiement au guichet enregistré par un agent
Chèque	CHEQUE	Paiement par chèque bancaire avec référence
Numérique / Mobile money	DIGI	Paiement via QOSIC, Fedapay ou PayGate (TMONEY / FLOOZ)

Les paiements sont toujours **rattachés à un Invoice** et donc à un **Taxpayer** (redevable). L'avis parcourt une **machine d'état Symfony Workflow** contrôlée avant de pouvoir être encaissé, et chaque paiement est **ventilé par code taxe** afin de permettre un reporting par ligne de recette.

2. Glossaire métier

Terme (FR / EN)	Signification
Avis / Invoice	Avis de mise en recouvrement émis à un redevable (Invoice)
Avis sur titre — TITRE	Avis formel nécessitant une approbation avant encaissement
Avis au comptant — COMPTANT	Avis immédiat émis sur place (sans exigence de numéro d'ordre)
Redevable / Taxpayer	Personne physique ou morale redevable de la taxe (Taxpayer)
Article taxable / TaxpayerTaxable	Ligne taxable rattachée à un redevable (TaxpayerTaxable)
Code	Code comptable identifiant une taxe / ligne de recette (TaxLabel.code)
Régisseur	Agent habilité à valider la chaîne de collecte
Receveur / Collector	Agent qui perçoit physiquement les fonds
Versement / Deposit	Remise de fonds du collecteur au comptable
Annulation / Réduction	Lignes de paiement « négatives » utilisées pour annuler ou réduire un avis approuvé

3. Architecture générale



4. Modèle de données

4.1 Invoice — app/Models/Invoice.php

Champs principaux :

Champ	Type	Rôle
invoice_no	chaîne (unique)	Numéro d'avis public
order_no	chaîne	Numéro d'ordre requis pour les avis TITRE pour passer en PENDING
taxpayer_id	FK	Redevable concerné
amount	float	Montant total facturé
reduce_amount	float	Montant après réduction (si réduit)
qty	float	Quantité (articles taxables)
from_date / to_date	date	Période de couverture
pay_status	chaîne	OWING · PART PAID · PAID
status	chaîne	État du workflow (voir §6)
type	chaîne	TITRE ou COMPTANT
delivery	chaîne	NOT DELIVERED / DELIVERED
delivery_date , delivery_to	date / chaîne	Informations de livraison
edition_state , printed_at	chaîne / datetime	Suivi de l'impression
notes	json	previous_invoice_id , remaining_amount , free_text
uuid	chaîne	Clé d'idempotence pour la synchronisation

Relations clés : taxpayer() , payments() , invoiceitems() , taxpayer_taxables() , printFiles() .

Méthodes clés (via InvoiceTrait + modèle) :

- can(\$state) / submitToState(\$state) — gardes et transitions du workflow.
- get_remains_to_be_paid() — montant restant à payer.
- isValid() — non rejeté / annulé / réduit et non déjà PAID.
- canGetPayment() — l'avis est dans un état permettant un paiement.
- sumAmountsByTaxCode(\$invoice) — totaux regroupés par code .
- getCode(\$id, \$amount, \$paymentData) — ventile un montant de paiement sur les codes taxe de l'avis (produit une ligne Payment par code).
- returnPaidAndSumByCode(\$invoice) — payé vs facturé par code.
- retrieveByUUIDs(array \$uuids) — chargement eager pour export / PDF.

Scopes utiles : ofStatus , approved , unpaid , forTaxpayer , inDateRange .

4.2 Payment — app/Models/Payment.php

Champs principaux :

Champ	Type	Rôle
amount	float	Montant payé
payment_type	chaîne enum	CASH · CHEQUE · DIGI
invoice_type	chaîne	Copie dénormalisée de Invoice.type
reference	chaîne	Référence du paiement (n° chèque, ref transaction, ...)
description	chaîne	ex. Annulation , Réduction , texte libre
remaining_amount	chaîne	Restant sur l'avis au moment du paiement
status	chaîne	PENDING · ACCOUNTED · DONE · CANCELED · VERIFYING · FAILED · EXPIRED
code	chaîne	Code taxe auquel cette ligne de paiement est imputée
taxpayer_id, invoice_id	FK	Cibles
user_id	FK	Utilisateur émetteur
r_user_id	FK	Utilisateur récepteur (collecteur)
deposit	float / ref	Référence du versement collecteur
notes	texte	Notes libres
uuid	chaîne	Clé d'idempotence pour la synchronisation
provider	chaîne	qosic · fedapay · paygate (DIGI uniquement)
network	chaîne	TMONEY · FLOOZ (DIGI uniquement)
phone_number, external_id	chaîne	DIGI uniquement
verification_attempts, last_checked_at, expires_at	mixte	Métadonnées de vérification DIGI

Relations : invoice(), taxpayer(), user(), r_user(), tax_label(), mobilePaymentTransaction(), stock_transfers() .

Méthodes du trait (PaymentTrait) :

- getPaymentsByStatus(\$invoice_id, \$status)
- getPaid(\$invoice_id) — somme des paiements effectifs
- getPaidNotAccounted(\$invoice_id) — versements en attente de comptabilisation
- getRestToPaid(Invoice \$invoice)
- getPrintData() — paiements pour le PDF (exclut PENDING , Annulation , Réduction)

Un paiement métier peut produire **plusieurs lignes Payment** — une par code taxe — via Invoice::getCode() .

4.3 MobilePaymentTransaction — app/Models/MobilePaymentTransaction.php

Trace la conversation avec un prestataire mobile-money pour une tentative DIGI . Une transaction peut produire plusieurs lignes Payment finales après vérification.

Champ	Rôle
reference (unique)	Référence interne envoyée au prestataire
invoice_id, taxpayer_id, user_id	Cibles
amount (decimal 15,2)	Montant demandé
phone_number	Téléphone du payeur
provider	qosic · fedapay · paygate
network	TMONEY · FLOOZ
external_id	Identifiant de transaction chez le prestataire
status	pending → verifying → success / failed / expired
verification_attempts, last_checked_at, verified_at, expires_at	Métadonnées de polling
provider_response (json)	Dernière réponse brute du prestataire
meta (json)	Ex. code taxe, métadonnées libres

Helpers : `isPending()`, `isVerifying()`, `isSuccess()`, `isFailed()`, `isExpired()`.

4.4 InvoiceItem — app/Models/InvoiceItem.php

`invoice_id`, `taxpayer_taxable_id`, `qty`, `amount`, `ii_tariff`, `ii_seize`. Lignes de détail agrégées pour calculer le total de l'avis.

4.5 InvoiceCodeBalance — app/Models/InvoiceCodeBalance.php

Vue matérialisée de « combien est dû / payé par code taxe » pour un avis. Maintenu par `InvoiceCodeBalanceService`. Clé unique sur (`invoice_id`, `code`).

Champ	Rôle
invoice_id, taxpayer_id, code, year	Identité
amount_billed	Somme des lignes d'avis pour ce code
amount_paid	Somme des paiements effectifs pour ce code (exclut Annulation / Réduction)
remaining_amount	Différence calculée
status	OWING · PAID
last_payment_at	Horodatage du dernier paiement applicable

4.6 Taxpayer — app/Models/Taxpayer.php

Contient `tnif`, `nif`, `name`, `mobilephone`, `telephone`, `email`, `address`. Utilisé pour l'identification sur les avis, les notifications SMS et les paiements numériques.

5. Énumérations

Toutes les énumérations se trouvent dans `app/Enums/` en tant qu'énumérations PHP 8.1 natives à valeurs chaîne, consommées via `->value` (pas de casts Eloquent — voir les notes du projet).

5.1 PaymentTypeEnums

```
CASH    = 'CASH'
CHEQUE  = 'CHEQUE'
DIGI    = 'DIGI'
```

5.2 PaymentStatusEnums

```
PENDING    paiement enregistré, non encore comptabilisé (état par défaut du DIGI avant vérification)
VERIFYING  DIGI : prestataire a accepté, polling en cours
ACCOUNTED  paiement comptabilisé dans un versement / grand livre
DONE       entièrement apuré
CANCELED   paiement annulé manuellement
FAILED     DIGI : rejet du prestataire / échec de vérification
EXPIRED    DIGI : transaction expirée sans confirmation
```

5.3 InvoicePayStatusEnums

```
OWING      = 'OWING'           // rien payé
PART_PAID  = 'PART PAID'       // partiellement payé
PAID       = 'PAID'           // entièrement payé
```

5.4 InvoiceStatusEnums (états du workflow)

```
DRAFT
P_ACCEPTED      = 'ACCEPTED WITHOUT ORDER NO'
ACCEPTED        = 'ACCEPTED'
REJECTED_BY_OR
PENDING
REJECTED
APPROVED
APPROVED_CANCELLATION = 'APPROVED-CANCELLATION-OR-REDUCTION'
CANCELED        = 'APPROVED-CANCELED'
REDUCED          = 'APPROVED-REDUCED'
```

5.5 Constantes — app/Helpers/Constants.php

```
PROVIDER_QOSIC    = 'qosic'
PROVIDER_FEDAPAY  = 'fedapay'
PROVIDER_PAYGATE  = 'paygate'
MOBILE_PROVIDERS  = [qosic, fedapay, paygate]

NETWORK_TMONEY    = 'TMONEY'
NETWORK_FLOOZ     = 'FLOOZ'
MOBILE_NETWORKS   = [TMONEY, FLOOZ]

ANNULATION = 'Annulation' // marqueur pour les lignes de paiement d'annulation
REDUCTION  = 'Réduction'  // marqueur pour les lignes de paiement de réduction

INVOICE_TYPE_TITRE      = 'TITRE'
INVOICE_TYPE_COMPTANT   = 'COMPTANT'
```

6. Machine d'état des avis (workflow)

Le cycle de vie d'un avis est piloté par une machine d'état Symfony déclarée dans [config/workflow.php](#) et enregistrée via [config/workflow_registry.php](#). Le nom du workflow est `invoice`.

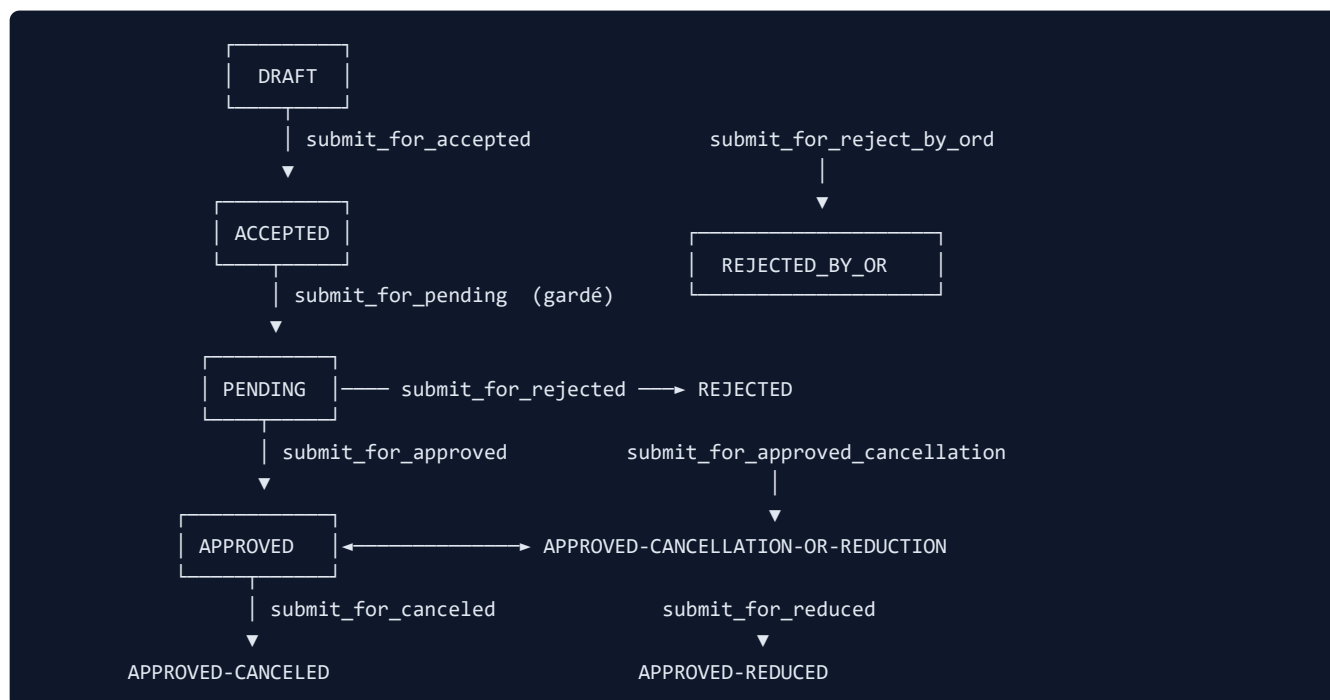
6.1 États (places)

```
DRAFT
ACCEPTED WITHOUT ORDER NO      (intermédiaire)
ACCEPTED
REJECTED_BY_OR                  (terminal)
PENDING
REJECTED                         (terminal)
APPROVED
APPROVED-CANCELLATION-OR-REDUCTION
APPROVED-CANCELED               (terminal)
APPROVED-REDUCED                (terminal)
```

6.2 Transitions

Transition	De	Vers	Garde
<code>submit_for_accepted</code>	DRAFT	ACCEPTED	—
<code>submit_for_reject_by_ord</code>	DRAFT	REJECTED_BY_OR	—
<code>submit_for_pending</code>	ACCEPTED	PENDING	<code>InvoiceGuard::canSubmitForPending</code> (<code>order_no</code> renseigné OU <code>type ==</code> <code>COMPTANT</code>)
<code>submit_for_approved</code>	PENDING	APPROVED	—
<code>submit_for_approved_cancellation</code>	PENDING	APPROVED- CANCELLATION-OR- REDUCTION	—
<code>submit_for_rejected</code>	PENDING	REJECTED	—
<code>submit_for_reduced</code>	APPROVED / APPROVED- CANCELLATION...	APPROVED- REDUCED	—
<code>submit_for_canceled</code>	APPROVED / APPROVED- CANCELLATION...	APPROVED- CANCELED	—

6.3 Diagramme de transitions



Les transitions sont observées par `InvoiceWorkflowSubscriber` qui diffuse les notifications et les post-actions (libération des articles taxables en cas de rejet, notification des agents, etc.).

Un paiement ne peut être ajouté que lorsque l'avis est en APPROVED (ou APPROVED-CANCELLATION-OR-REDUCTION).
Vérifié par `Invoice::canGetPayment()` et `PaymentPolicy`.

7. Cycle de vie d'un paiement



CANCELED est utilisé lorsqu'une ligne de paiement est annulée manuellement. Les paiements avec `description = "Annulation"` ou "Réduction" ne sont pas comptés comme paiements effectifs par `getPrintData()` et `InvoiceCodeBalanceService`.

8. Procédure de paiement en espèces ou par chèque

Composant : [app/Livewire/Payment/AddPaymentModal.php](#).

1. L'agent ouvre la fiche d'un redevable ou d'un avis.
2. Déclenche le **AddPaymentModal** (`payment_type` = CASH ou CHEQUE).
3. Champs requis :
 - `amount` (≤ montant restant sur l'avis).
 - `payment_type` ∈ { CASH , CHEQUE }.
 - `reference` (recommandé pour CHEQUE).
 - `notes` (optionnel).
4. À la validation :
 1. Le montant est **ventilé par code taxe** via `Invoice::getCode($invoice_id, $amount, $paymentData)`.
 2. Une ligne `Payment` est créée **par code taxe**, toutes partageant la même `reference` et `created_at`.
 3. `Invoice.pay_status` est recalculé : OWING → PART PAID → PAID via `getRestToPaid()`.
 4. `InvoiceCodeBalanceService::syncForInvoice($invoice)` est appelé pour mettre à jour les totaux par code.
 5. Si entièrement payé, la notification `InvoicePaid` est émise.
5. L'agent peut immédiatement imprimer le reçu PDF (voir §11).

Cycle de vie d'une ligne CASH/CHÈQUE :

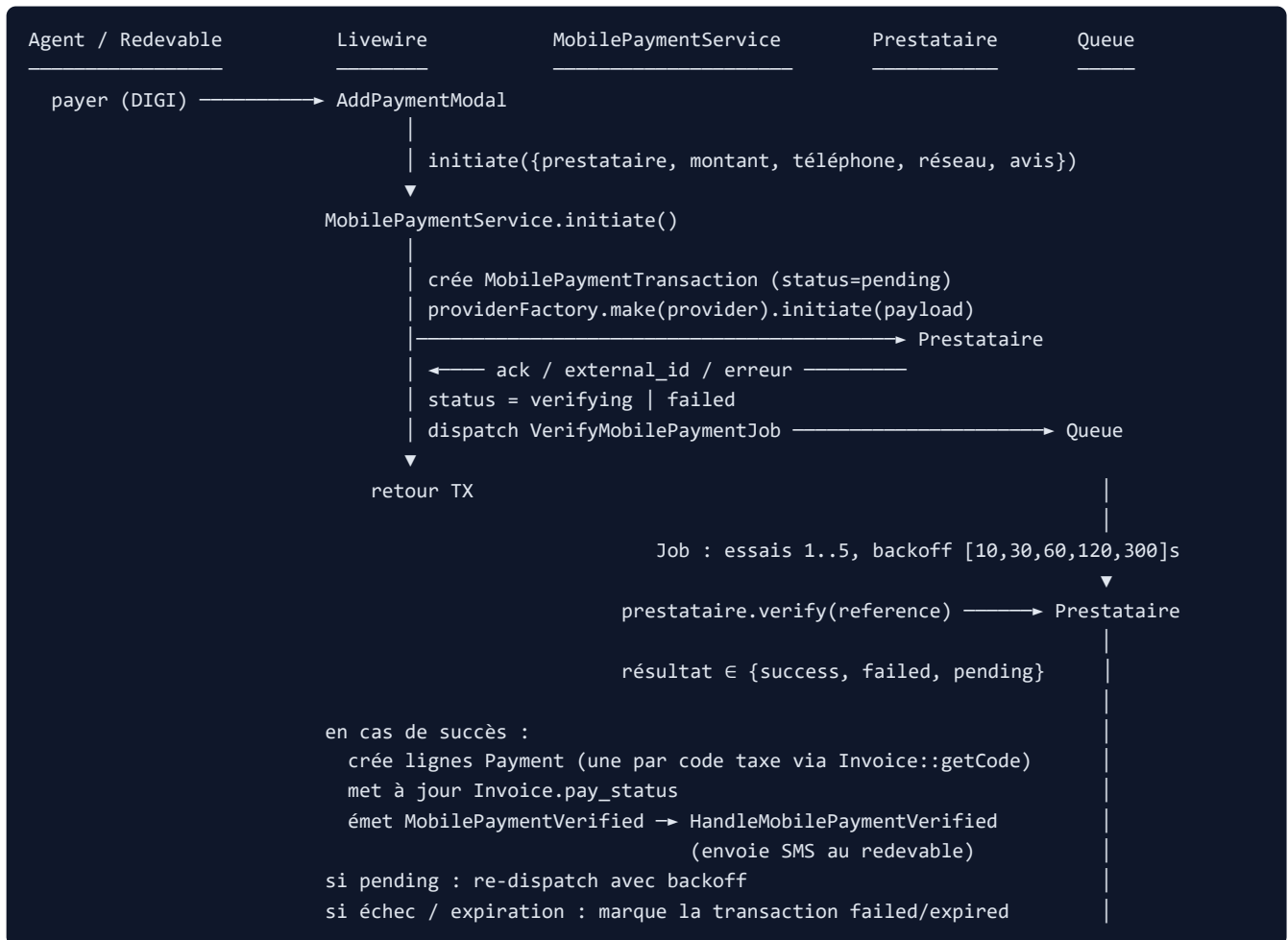
PENDING (créé) → ACCOUNTED (ajouté à un versement) → DONE (clôturé par le comptable)

CANCELED est accessible manuellement si la politique l'autorise.

9. Procédure de paiement mobile / numérique

La logique de bout en bout se trouve dans [app/Services/MobilePayment/MobilePaymentService.php](#). Elle est **basée sur la vérification active, jamais sur un webhook de confiance**.

9.1 Séquence



9.2 Données validées par AddPaymentModal

- amount requis, $\leq$ restant.
- payment_type = DIGI.
- phone_number — 8 chiffres.
- network $\in$ MOBILE_NETWORKS (TMONEY, FLOOZ).
- provider $\in$ MOBILE_PROVIDERS (qosic, fedapay, paygate).

9.3 Abstraction prestataire

MobilePaymentProviderInterface expose :

```

public function initiate(array $data): array; // retourne ['ok'=>bool, 'external_id'=>?, 'response'=>...]
public function verify(string $reference): array; // retourne ['status'=>'success|failed|pending', ...]
  
```

Implémentations concrètes :

- QosicProvider
- FedapayProvider
- PaygateProvider

La fabrique est MobilePaymentProviderFactory.

9.4 Politique de relance et d'expiration

Définie dans `config/mobile-payment.php` :

```
transaction_expiry_minutes = 1           # par défaut
verification.max_attempts  = 3
verification.backoff_seconds = [10, 30, 60, 120, 300]
```

`VerifyMobilePaymentJob` effectue 5 tentatives avec le backoff ci-dessus. Les transactions expirées sont balayées par `ExpireStateMobilePaymentsJob`.

9.5 Relance manuelle

`MobilePaymentActions` expose une action `retryVerification($id)` (visible dans l'écran **Paiements mobiles**) pour les transactions bloquées en `pending` / `verifying`.

10. Suivi des soldes par code taxe

Service : `app/Services/Sync/InvoiceCodeBalanceService.php`.

À chaque modification d'un avis (émission, paiement, annulation, réduction), `syncForInvoice($invoice)` :

1. Agrège `amount_billed` par `code` depuis `invoice_items` jointés à `taxpayer_taxables` / `tax_labels`.
2. Agrège `amount_paid` par `code` depuis `payments`, **en excluant** les lignes dont `description` est `Annulation` ou `Réduction`.
3. Met à jour ou crée une ligne `invoice_code_balances` par `(invoice_id, code)` avec `remaining_amount` et `status` (`OWING` / `PAID`).
4. Supprime les lignes de solde obsolètes pour les codes qui ne figurent plus dans l'avis.

Cette table est la source de vérité pour le reporting par code (grands livres, versements comptable, vues de recouvrement).

11. Génération de reçus et de PDF

Service : `app/Services/PdfGeneratorService.php` (utilise `Barryvdh\DomPDF\Facade\Pdf`).

API publique :

```
PdfGeneratorService::generateInvoicePdf(
    array $uuids,           // un ou plusieurs UUID d'avis
    string $templateName,   // resources/views/exports/{templateName}.blade.php
    ?int $action = null,    // 1 = notice, 2 = reçu
    bool $is_relance = false
): array // ['success'=>bool, 'pdf'=>stream, 'filename'=>string]
```

Comportement :

- Charge les avis en eager loading via `Invoice::retrieveByUUIDs()`.
- Ajoute un QR code via `QrcodeGeneratorService` — utilisé par les collecteurs pour la réconciliation sur le terrain.
- Lors de la première génération, renseigne `edition_state = 'PRINT'` et `printed_at = now()`.
- Les templates se trouvent dans `resources/views/exports/` (ex. `invoices.blade.php`).

Routes d'impression :

- `GET /prints` — interface de file d'attente d'impression.
- `GET /print-all-invoice` — téléchargement PDF en masse (`PrintController@downloadMultipleInvoicePdf`).

12. Synchronisation avec l'application de collecte distante

Une application mobile compagnon collecte les paiements sur le terrain et synchronise avec SIG-RECETTE. Voir [docs/sync-entites-paiements.md](#) et [docs/sync-implementation.md](#).

12.1 Entités et sens de synchronisation

Entité	Sens	Objet
Invoice	local → distant	L'app distante affiche les avis impayés
InvoiceItem	local → distant (lecture seule)	Lignes de détail
Taxpayer	local → distant (lecture seule)	Identification & contact
TaxpayerTaxable	local → distant (lecture seule)	Contexte des articles taxables
Payment	distant → local	Paiements collectés sur le terrain remontés vers le local

12.2 Points d'API — routes/api.php

```

GET /api/v1/sync/invoices      → Api/Sync/V1/SyncV1InvoicesController@index
GET /api/v1/sync/payments     → Api/Sync/V1/SyncV1PaymentsController@index (export paiements locaux)
POST /api/v1/sync/payments    → Api/Sync/V1/SyncV1PaymentsController@store (import paiements distants)

POST /api/v1/search/invoices  → SearchInvoiceController@search
POST /api/v1/search/taxpayers → SearchTaxpayerController@search
POST /api/v1/search/taxpayer-taxables → SearchTaxpayerTaxableController@search

```

12.3 Idempotence et filtres

- Idempotent via `uuid` sur `Invoice` et `Payment` (UUID v4 généré à la création).
- Incrémental via `?updated_since=<ISO 8601>`.
- Filtrage par année active via `Year::getActiveYear()`.
- Pagination via `?per_page=`.

12.4 Service d'import

PaymentImportService :

```

PaymentImportService::import(array $payments): [
    'created' => int,
    'updated' => int,
    'skipped' => int,
    'errors' => array,
]

```

Pour chaque entrée :

1. Résolution de l'avis via `invoice_uuid`.
2. `Payment::firstOrCreate(['uuid' => ...], $attributes)` puis mise à jour si existant.
3. Recalcul de `Invoice.pay_status` et appel à `InvoiceCodeBalanceService`.

12.5 Jobs en arrière-plan

- `SyncImportPaymentsJob` — récupère les paiements distants page par page (délai 120s, 3 tentatives).

- `SyncExportInvoicesJob` — pousse les avis vers le distant.

Les exécutions de synchronisation sont tracées dans les enregistrements `SyncRun` utilisés comme curseur.

13. Événements, écouteurs, jobs et notifications

13.1 Événements

Événement	Objet
<code>MobilePaymentVerified</code>	Émis lorsqu'une <code>MobilePaymentTransaction</code> passe en <code>success</code>

13.2 Écouteurs

Écouteur	Réagit à	Action
<code>HandleMobilePaymentVerified</code>	<code>MobilePaymentVerified</code>	Envoie le SMS « <i>Votre paiement de {montant} FCFA pour l'avis {invoice_no} a été confirmé. Réf : {reference}</i> » (conditionné par <code>features.sms_notifications_feature</code>)
<code>InvoiceWorkflowSubscriber</code>	<code>workflow.invoice.</code> {enter,leave,transition,guard}	Notifie les utilisateurs concernés (<code>InvoiceAccepted</code> , <code>InvoiceApproved</code> , <code>InvoiceRejected</code>), libère les <code>TaxpayerTaxable</code> réservés en cas de rejet, etc.

13.3 Jobs

Job	Objet
<code>VerifyMobilePaymentJob</code>	Interroge le prestataire — 5 tentatives, backoff [10, 30, 60, 120, 300] s
<code>ExpireStaleMobilePaymentsJob</code>	Marque comme <code>expired</code> les transactions dont <code>expires_at</code> est dépassé
<code>SyncImportPaymentsJob</code>	Récupère les paiements depuis le distant
<code>SyncExportInvoicesJob</code>	Pousse les avis vers le distant

13.4 Notifications (canal base de données)

`InvoiceCreated` , `InvoiceAccepted` , `InvoiceApproved` , `InvoiceRejected` , `InvoicePaid` — toutes dans [app/Notifications/](#).

14. Autorisations et permissions

Les chaînes de permission sont en **français** et gérées par Spatie.

14.1 InvoicePolicy — app/Policies/InvoicePolicy.php

Capacité	Permission requise
viewAny / view	Tout utilisateur authentifié
create	<i>peut émettre un avis au comptant OU peut émettre un avis sur titre</i>
update	<i>peut accepter un avis sur titre, peut prendre en charge..., peut ajouter date de livraison, peut ajouter numéro d'ordre</i>
delete	<i>peut rejeter un avis (et variantes déléguées)</i>
reduce	<i>peut réduire un avis (titre / comptant)</i>

14.2 PaymentPolicy — app/Policies/PaymentPolicy.php

Capacité	Permission requise
viewAny / view	Tout utilisateur authentifié
create	<i>peut ajouter un paiement</i>
update	<i>peut accepter un paiement OU peut comptabiliser un paiement</i>
delete	<i>peut supprimer un paiement</i>

14.3 InvoiceGuard — app/Guards/InvoiceGuard.php

canSubmitForPending bloque ACCEPTED → PENDING sauf si order_no est renseigné ou type == COMPTANT.

15. Référence des routes

15.1 Web — routes/web.php

```

GET /invoices InvoiceController@index
GET /invoices/{id} InvoiceController@show
GET /mobile-payments MobilePaymentController@index

GET /accounts/collector-deposits CollectorDepositController@index
GET /accounts/collector-deposits/{id} CollectorDepositController@show
GET /accounts/accountant-deposits-title AccountantDepositController@index
GET /accounts/accountant-deposits-outright AccountantDepositOutrightController@index
GET /accounts/ledgers LedgerController@index

GET /recoveries RecoveryController@index
GET /prints PrintController@index
GET /print-all-invoice PrintController@downloadMultipleInvoicePdf

```

15.2 API — routes/api.php

```
GET /api/v1/sync/invoices      Api/Sync/V1/SyncV1InvoicesController@index
GET /api/v1/sync/payments     Api/Sync/V1/SyncV1PaymentsController@index
POST /api/v1/sync/payments    Api/Sync/V1/SyncV1PaymentsController@store

POST /api/v1/search/invoices  SearchInvoiceController@search
POST /api/v1/search/taxpayers SearchTaxpayerController@search
POST /api/v1/search/taxpayerstaxables SearchTaxpayerTaxableController@search
```

16. Référence de configuration

16.1 config/mobile-payment.php

```
'default_provider' => env('MOBILE_PAYMENT_DEFAULT_PROVIDER', 'paygate'),
'transaction_expiry_minutes' => env('MOBILE_PAYMENT_EXPIRY_MINUTES', 1),

'verification' => [
    'max_attempts' => 3,
    'backoff_seconds' => [10, 30, 60, 120, 300],
],

'providers' => [
    'qosic' => ['base_url'=>env('QOSIC_BASE_URL'), 'username'=>env('QOSIC_USERNAME'), 'password'=>env('QOSIC_PASSWORD'), 'merchant_id'=>env('QOSIC_MERCHANT_ID')],
    'fedapay' => ['base_url'=>env('FEDAPAY_BASE_URL', 'https://sandbox-api.fedapay.com'), 'secret_key'=>env('FEDAPAY_SECRET_KEY'), 'public_key'=>env('FEDAPAY_PUBLIC_KEY')],
    'paygate' => ['base_url'=>env('PAYGATE_BASE_URL'), /* ... */ ],
],
```

16.2 config/workflow.php / config/workflow_registry.php

Déclarent la machine d'état `invoice` (états et transitions listés au §6).

16.3 config/sync.php

Contient les endpoints, tokens et paramètres de pagination pour l'application de collecte distante.

16.4 Clés `.env` requises (liées au paiement)

```
MOBILE_PAYMENT_DEFAULT_PROVIDER=paygate
MOBILE_PAYMENT_EXPIRY_MINUTES=1

QOSIC_BASE_URL=
QOSIC_USERNAME=
QOSIC_PASSWORD=
QOSIC_MERCHANT_ID=

FEDAPAY_BASE_URL=https://sandbox-api.fedapay.com
FEDAPAY_SECRET_KEY=
FEDAPAY_PUBLIC_KEY=

PAYGATE_BASE_URL=
PAYGATE_*=...

QUEUE_CONNECTION=database # requis pour VerifyMobilePaymentJob
```


17. Référence des migrations de base de données

17.1 `invoices` (migration ancre)

[2024_02_03_000548_create_invoices_table.php](#)

```
id          BIGINT PK AUTOINC (démarré à 100001)
invoice_no   VARCHAR
order_no     VARCHAR
status       VARCHAR DEFAULT 'DRAFT'
pay_status   VARCHAR DEFAULT 'OWING'
delivery     VARCHAR DEFAULT 'NOT DELIVERED'
delivery_date DATE
from_date    DATE
to_date      DATE
qty          DOUBLE
amount       DOUBLE DEFAULT 0
taxpayer_id  FK → taxpayers.id
created_at, updated_at
```

Les migrations progressives ajoutent `validity`, `reduce_amount`, `delivery_to`, `uuid`, `type`, `is_on_recovery_print`, `edition_state`, `notes` (json), `reason_for_reject`, `printed_at`.

17.2 `payments`

[2024_02_26_005552_create_payments_table.php](#) — colonnes de base, puis migrations incrémentales ajoutent `remaining_amount`, `user_id`, `uuid`, `code`, `invoice_type`, `notes`, référence de versement ; enfin :

- [2026_03_28_000001_add_mobile_payment_fields_to_payments_table.php](#) — `provider`, `phone_number`, `external_id`, `verification_attempts`, `last_checked_at`, `expires_at`.
- [2026_04_08_000001_add_network_column_to_payments_and_mobile_payment_transactions_tables.php](#) — `network`.

17.3 `mobile_payment_transactions`

[2026_03_28_000002_create_mobile_payment_transactions_table.php](#)

```
id, reference UNIQUE,
invoice_id FK, taxpayer_id FK NULL, user_id FK NULL,
amount DECIMAL(15,2), phone_number, provider, network,
external_id, status DEFAULT 'pending',
verification_attempts SMALLINT DEFAULT 0,
last_checked_at, verified_at, expires_at,
provider_response JSON, meta JSON,
INDEX (invoice_id, status), INDEX (status, expires_at), INDEX (provider)
```

17.4 `invoice_code_balances`

[2026_03_01_000000_create_invoice_code_balances_table.php](#)

```

invoice_id FK, taxpayer_id FK,
code, year,
amount_billed, amount_paid, remaining_amount,
status DEFAULT 'OWING', last_payment_at,
UNIQUE (invoice_id, code)
INDEX (taxpayer_id, code, year)
INDEX (taxpayer_id, year)

```

18. Procédures étape par étape

18.1 Émettre un avis et encaisser un paiement en espèces (COMPTANT)

1. **Ouvrir la fiche d'un redevable** → cliquer *Nouvel avis*.
2. Remplir le `AddInvoiceModal` (redevable, articles, période). Choisir `type = COMPTANT`.
3. Valider → avis créé en `DRAFT`.
4. Appliquer les transitions :
 - `submit_for_accepted` (`DRAFT` → `ACCEPTED`).
 - `submit_for_pending` (`ACCEPTED` → `PENDING`) — **autorisé parce que** `type = COMPTANT` même sans `order_no`.
 - `submit_for_approved` (`PENDING` → `APPROVED`).
5. Ouvrir `AddPaymentModal`, choisir `CASH`, saisir le montant, valider.
6. Le système crée une ligne `Payment` par code taxe, recalcule `pay_status` et actualise `invoice_code_balances`.
7. Imprimer le reçu (\$11).

18.2 Émettre un avis sur titre (TITRE) — numéro d'ordre obligatoire

1. Idem ci-dessus avec `type = TITRE`.
2. Après `ACCEPTED`, l'agent **doit** renseigner `order_no` (permission *peut ajouter numéro d'ordre*).
3. `submit_for_pending` réussit alors ; sinon `InvoiceGuard::canSubmitForPending` bloque la transition.
4. Poursuivre avec l'approbation et le paiement.

18.3 Encaisser un paiement numérique (DIGI)

1. Ouvrir `AddPaymentModal` sur un avis en `APPROVED`.
2. Choisir `DIGI`, sélectionner le prestataire (`paygate` par défaut), le réseau (`TMONEY / FLOOZ`), saisir le **numéro à 8 chiffres**.
3. Valider → `MobilePaymentService::initiate()` crée une `MobilePaymentTransaction` (`pending`), appelle le prestataire et met en file un `VerifyMobilePaymentJob`.
4. Le redevable valide la demande USSD/application sur son téléphone.
5. Le job interroge le prestataire avec backoff. En cas de `success`, les lignes `Payment` sont créées, `pay_status` mis à jour, l'événement `MobilePaymentVerified` est émis et le redevable reçoit un SMS de confirmation.
6. Si le polling épuise les tentatives ou que `expires_at` est dépassé, la transaction est marquée `failed` / `expired`. L'agent peut utiliser *Réessayer* depuis la page **Paiements mobiles**.

18.4 Annuler ou réduire un avis approuvé

1. Depuis `APPROVED`, transition `submit_for_approved_cancellation` → `APPROVED-CANCELLATION-OR-REDUCTION`.
2. Puis :
 - `submit_for_canceled` → `APPROVED-CANCELED`, **et** créer un paiement correctif avec `description = "Annulation"`.
 - `submit_for_reduced` → `APPROVED-REDUCED`, **et** créer un paiement correctif avec `description = "Réduction"` et mettre à jour `Invoice.reduce_amount`.
3. Ces lignes `Annulation` / `Réduction` sont exclues de `getPrintData()` et du calcul du solde par code.

18.5 Clôturer un versement collecteur

1. Aller dans *Comptes* → *Versements régisseur* (`/accounts/collector-deposits`).

2. Sélectionner la plage de dates ; le système liste les lignes `Payment` avec `r_user_id = collecteur courant` et `status = PENDING`.
3. Valider le versement : `status` passe à `ACCOUNTED`, la colonne `deposit` est renseignée.
4. Les versements comptable sont ensuite clôturés sous *Versements comptable* (titre / comptant), ce qui les fait passer à `DONE`.

18.6 Synchroniser avec l'application de collecte distante

1. Planifier `SyncExportInvoicesJob` afin que le distant dispose toujours des avis `APPROVED` à jour.
2. Planifier `SyncImportPaymentsJob` toutes les X minutes — il appelle `GET /api/v1/sync/payments?updated_since=...&per_page=...`, puis `PaymentImportService::import()`.
3. Le curseur (`updated_since`) est persisté via `SyncRun`.

19. Résolution des problèmes

Symptôme	Cause probable	Où chercher
Impossible de faire passer l'avis de <code>ACCEPTED</code> à <code>PENDING</code>	<code>order_no</code> absent sur un avis <code>TITRE</code>	<code>InvoiceGuard::canSubmitForPending</code>
<code>AddPaymentModal</code> rejette le montant	<code>Montant > Invoice::get_remains_to_be_paid()</code>	Règles de validation dans <code>AddPaymentModal</code>
DIGI bloqué en <code>verifying</code>	Queue non démarrée ou délai d'attente prestataire	Lancer <code>php artisan queue:work</code> ; inspecter <code>mobile_payment_transactions.last_checked_at</code> , <code>provider_response</code> ; cliquer <i>Réessayer</i>
DIGI finit en <code>expired</code>	<code>transaction_expiry_minutes</code> trop court ou redevable n'a pas validé	<code>config/mobile-payment.php</code> , <code>ExpireStaleMobilePaymentsJob</code>
Soldes par code obsolètes	<code>InvoiceCodeBalanceService::syncForInvoice()</code> non appelé	Déclencher en enregistrant un <code>Payment</code> sur l'avis ; ou resynchroniser via <i>tinker</i>
Import sync indique <code>skipped</code>	Même <code>payment.uuid</code> déjà importé (idempotence)	Comportement attendu
Le reçu PDF affiche des lignes <code>Annulation</code>	Ces lignes sont normalement masquées ; vérifier le filtre <code>description</code> dans <code>Payment::getPrintData()</code>	<code>app/Traits/PaymentTrait.php</code>
Notification jamais envoyée	<code>features.sms_notifications_feature</code> désactivé ou identifiants SMS absents	<code>config/features.php</code> , <code>SmsService</code>

Annexe — Aide-mémoire de référence rapide

```

États avis      : DRAFT → ACCEPTED → PENDING → APPROVED → (CANCELED|REDUCED)
Statut paiement: OWING → PART PAID → PAID
Types paiement : CASH | CHEQUE | DIGI
Prestataires   : qosic | fedapay | paygate
Réseaux mobiles: TMONEY | FLOOZ
Statut TX DIGI : pending → verifying → success | failed | expired
Statut Payment : PENDING → ACCOUNTED → DONE (CANCELED, + DIGI : VERIFYING/FAILED/EXPIRED)
Clé idempotence: uuid (Invoice & Payment)
Soldes/code     : invoice_code_balances (unique sur invoice_id+code)

```

Document généré pour le projet SIG-RECETTE.